

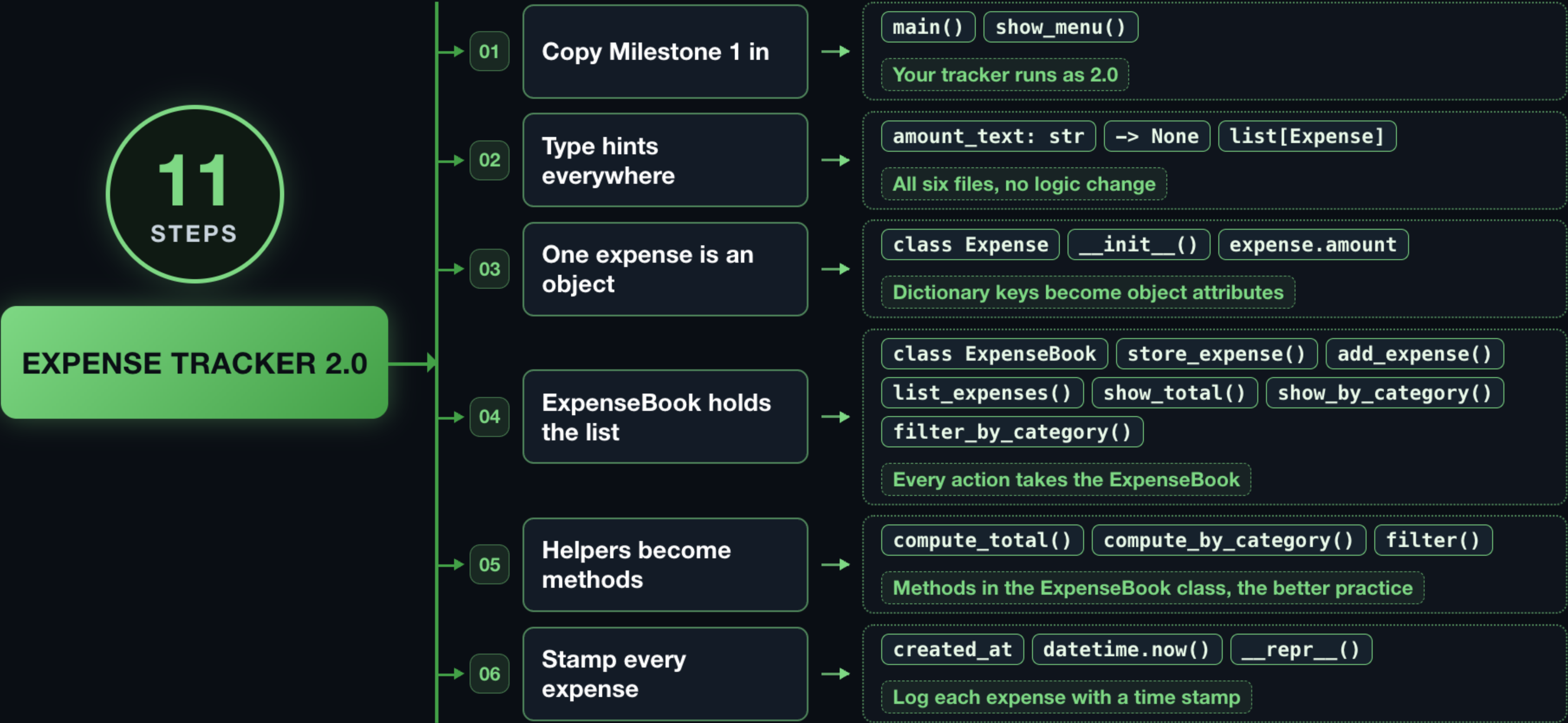
Milestone Project-2: Upgrade CLI Expense Tracker to Persistent Expense Tracker

The same Expense Tracker and the same menu. Milestone 2 changes how it works inside, using Sections 08 to 13.

	MILESTONE 1: DICTIONARY-BASED	UPGRADE	MILESTONE 2: OBJECTS + JSON + ERROR HANDLING
One expense	a dictionary with three keys	Section 13 →	an <code>Expense</code> object with attributes
Who owns the list	<code>main()</code> holds <code>expenses = []</code>	Section 13 →	<code>ExpenseBook</code> owns it: <code>book = ExpenseBook(...)</code>
Read a value	<code>expense["amount"]</code>	Section 13 →	<code>expense.amount</code>
Total spent	<code>compute_total(expenses)</code>	Section 13 →	<code>book.compute_total()</code>
By category	<code>compute_by_category(expenses)</code>	Section 13 →	<code>book.compute_by_category()</code>
One category	a list comprehension inside <code>filter_by_category()</code>	Section 13 →	<code>book.filter()</code> , called by the same action
Types	<i>no type hints</i>	Section 08 →	<code>amount_text: str, -> bool, list[Expense]</code>
Quit and restart	everything is gone	Section 10 →	<code>book.save()</code> and <code>book.load(): data/expenses.json</code>
A broken file	<i>no file, so nothing to break</i>	Section 09 →	<code>try / except</code> : a warning, no crash
New answers	<i>not there yet</i>	Section 11 →	<code>created_at</code> time stamp on every expense
One package	<i>nothing but Python</i>	Section 12 →	<code>.venv</code> and <code>pip install rich</code> : a real table (<i>bonus</i>)

Milestone Project-2: CLI Persistent Expense Tracker - What You Will Build? 1 of 2

Eleven steps build it. Steps 01 to 06 turn your Milestone 1 code into objects and a book.



Milestone Project-2: CLI **Persistent** Expense Tracker - What You Will **Build?** 2 of 2

Steps 07 to 11 give it a file, a guard, a full run, and the optional rich bonus.



Project 2: How Your Data Survives

Project 1 forgot everything when you quit. Project 2 writes a file, so it comes back.

SESSION 1: TODAY

- 1 Add **Coffee 3.50 food**
- 2 It lives in **memory** only
- 3 Choose **6) Save and quit**
- 4 `save()` writes the file



program ends
memory wiped

SESSION 2: TOMORROW

- 1 Start the program again
- 2 `load()` reads the file
- 3 Objects are **rebuilt**
- 4 Coffee is **still there**

MEMORY: WHAT DIES

The ExpenseBook and its objects. Fast, but **gone the moment the program stops**.
Every new run starts with an empty list, until `load()` fills it again.

DISK: WHAT STAYS

`expenses.json` is plain text. It **stays until you delete it**.
Written by `save()` on choice 6, read back the next time you start.

WHERE IT LIVES

`Path(__file__).parent / "data" / "expenses.json"`
A data folder next to your code, not in the folder you ran from.

Saving

Expense object

→
`read_attributes`

plain dict

→
`json.dumps()`

JSON text

→
`write_text()`

file on disk

Loading

file on disk

→
`read_text()`

JSON text

→
`json.loads()`

plain dict

→
`Expense(row[...])`

Expense object

Project 2: Which Folder Is Which

Five folders in `milestone-projects/milestone-project-2/`. One of them is yours to type in.

`01-orig-build-incremental`

THE ANSWER

The finished tracker, `Steps 01 to 10`. Your answer key.

`02-orig-build-incremental-with-rich`

BONUS ANSWER

After `Step-11`, with the `rich` table. Your bonus answer key.

`03-orig-warmup`

OPTIONAL

Six short drills, one new skill each. An optional extra.

`04-livecode-build-incremental`

YOU BUILD HERE

Your **Milestone 1** code. Build `Steps 01 to 10` here.

`05-livecode-build-incremental-with-rich`

BONUS

The finished `Step-11` code. Read, run, make the venv.

IN SHORT

Everything you type goes in `04-livecode-build-incremental`. Compare with `01` when you are done.

Step-01: Copy Milestone 1 In, Retitle the Menu in **main.py**

Start from the Milestone 1 files in 04-livecode-build-incremental, and change one line so the menu says 2.0.

CONCEPTS USED

Run a program: python3
Section 02

Output: print()
Section 02

WHAT WE BUILD

04-livecode-build-incremental/
the six Milestone 1 files, no comments

Menu title
==== Expense Tracker 2.0 ====

Second run
prints No expenses yet.

main.py
your Milestone 1 code

Step-02: Add Type Hints, Starting with `f1_is_valid_amount.py`

Type hints on every function in all six files. No logic changes, and the output stays the same.

CONCEPTS USED

Type hints: parameters and `->`
Section 08

Type hints: `list[...]`, `dict[...]`
Section 08

Type hints: not enforced
Section 08

Type hints: `-> None`
New in this project

**`f1_is_valid_`
`amount.py`
and five more files**

WHAT WE BUILD

`is_valid_amount()`
takes `str`, returns `bool`

`format_row()`
takes `dict`, returns `str`

`compute_total()`
takes `list[dict]`, returns `float`

`compute_by_category()`
returns `dict[str, float]`

Five actions
each returns `None`

`show_menu()` and `main()`
both return `None`

Choice 3 output
Total spent: 3.50, as before

Step-03: One Expense Dictionary Becomes an Object

A dictionary with **three keys** becomes an Expense object with **three attributes**. Every square-bracket read becomes a dot.

BEFORE: A DICTIONARY

Milestone 1

One expense

```
expense = {  
    "description": description,  
    "amount": amount,  
    "category": category,  
}
```

Read a value

```
desc = expense["description"]  
cat = expense["category"]  
amt = expense["amount"]
```

The expense list

```
expenses: list[dict] = []
```

AFTER: AN EXPENSE OBJECT

Milestone 2, new file expense.py

One expense

```
class Expense:  
    def __init__(self, description: str, amount: float, category: str) -> None:  
        self.description = description  
        self.amount = amount  
        self.category = category
```

```
expense = Expense(description, amount, category)
```

Read a value

```
desc = expense.description  
cat = expense.category  
amt = expense.amount
```

The expense list

```
expenses: list[Expense] = []
```

The same hint changes in every function that takes the list: f3, f4 and all five actions in f5.

Step-03 touches six files

expense.py new

main.py

f5_actions.py

f2_format_row.py

f3_compute_total.py

f4_compute_by_category.py

Step-03: 3.1 One Expense Becomes an Object in `expense.py`

`Expense` describes one expense: `__init__` stores its description, amount and category on the object.

CONCEPTS USED

Classes: a blueprint for objects

Section 13

Classes: `__init__` and `self`

Section 13

Type hints: parameters and `->`

Section 08

WHAT WE BUILD

`expense.py`

new file: home of the `Expense` class

`class Expense`

one expense, as an object

`__init__()`

stores description, amount, category

Read with a dot

`expense.amount`, later in f2 to f5

`expense.py`

the `Expense` class

Step-03: 3.3 to 3.7 Build It, Then Fix Every Read in `f5_actions.py`

`add_expense()` builds an `Expense`, and the `TypeError` shows each read to change into a dot, until every choice prints the same output.

CONCEPTS USED

Modules: `from ... import`
Section 07

Classes: attributes with a dot
Section 13

Debugging: read the traceback
Section 09

`f5_actions.py`
then `f2`, `f3` and `f4`

WHAT WE BUILD

`f5_actions.py`
`add_expense()` builds an `Expense`

`TypeError`
'Expense' object is not subscriptable

`f2_format_row.py`
three reads become dots

`f3_compute_total.py`
one read becomes a dot

`f4_compute_by_category.py`
two reads become dots

`f5_actions.py`
`filter_by_category()`, one read

Step-04: The Expense **List** Becomes an **ExpenseBook**

`main()` stops holding the list itself. An **ExpenseBook** owns the list, stores each new expense, and **every action takes the book**.

BEFORE: MAIN() HOLDS THE LIST

end of Step-03

Who holds the list `main.py` in `main()`

```
def main() -> None:
    expenses: list[Expense] = []
```

Store one expense `f5_actions.py` in `add_expense()`

```
expense = Expense(description, amount, category)
expenses.append(expense)
```

Hand it to the actions `main.py` in `main()`

```
add_expense(expenses)
list_expenses(expenses)
```

Inside an action `f5_actions.py` in `list_expenses()`

```
def list_expenses(expenses: list[Expense]) -> None:
    for expense in expenses:
```

AFTER: THE BOOK HOLDS THE LIST

new file `expense_book.py`

Who holds the list `expense_book.py` in `__init__()`

```
class ExpenseBook:
    def __init__(self) -> None:
        self.expenses: list[Expense] = []
```

Store one expense `expense_book.py` in `store_expense()`

```
def store_expense(self, description: str, amount: float, category: str) -> Expense:
    expense = Expense(description, amount, category)
    self.expenses.append(expense)
    return expense
```

Hand it to the actions `main.py` in `main()`

```
book = ExpenseBook()
add_expense(book)
list_expenses(book)
```

Inside an action `f5_actions.py` in `list_expenses()`

```
def list_expenses(book: ExpenseBook) -> None:
    for expense in book.expenses:
```

What changes `expense_book.py` **new** `class ExpenseBook` with `store_expense()` `main.py` `main()` makes the book and passes it to every action

`f5_actions.py` all five actions take the book: `add_expense()` `list_expenses()` `show_total()` `show_by_category()` `filter_by_category()`

Step-04: 4.1 ExpenseBook Holds the List in `expense_book.py`

`ExpenseBook` keeps the list inside the class, and `store_expense()` builds and keeps each `Expense`.

CONCEPTS USED

Classes: `__init__` and `self`
Section 13

Classes: methods
Section 13

Type hints: `list[...]`, `dict[...]`
Section 08

`expense_book.py`
the `ExpenseBook` class

WHAT WE BUILD

`ExpenseBook`
holds `self.expenses`

`self.expenses`
starts as an empty list

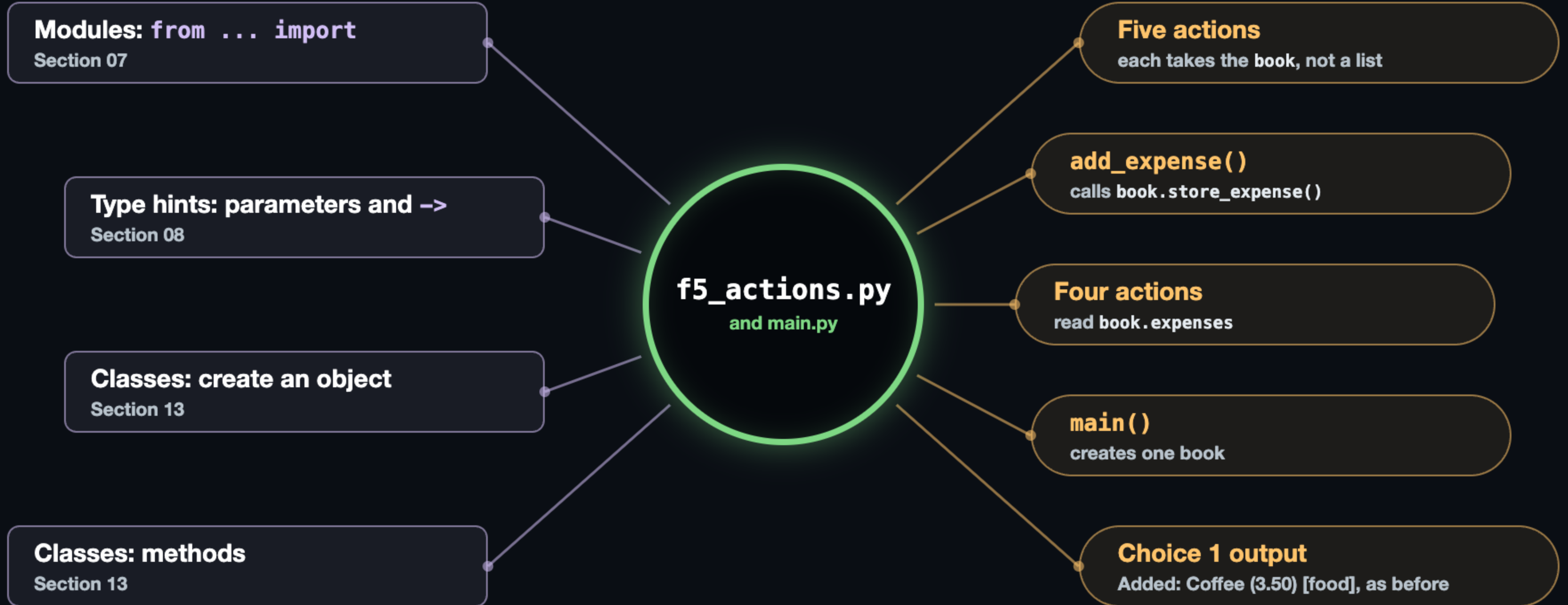
`store_expense()`
builds, keeps and returns an `Expense`

Step-04: 4.2 to 4.7 Every Action Takes the Book in `f5_actions.py`

The five actions take the **book** instead of a list, and **main()** creates one book and passes it on.

CONCEPTS USED

WHAT WE BUILD



Step-05: Why Move Two **Helpers** Into the **ExpenseBook**?

The **ExpenseBook** owns the expenses, so the **ExpenseBook** does the work on them.

BEFORE: THE WORK IS SPREAD OUT end of Step-04

receives the list `f3_compute_total.py`

```
def compute_total(expenses: list[Expense])
```

receives the list `f4_compute_by_category.py`

```
def compute_by_category(expenses: list[Expense])
```

searches the list on its own `f5_actions.py` `filter_by_category()`
for expense in `book.expenses`



AFTER: THE BOOK DOES ITS OWN WORK end of Step-05

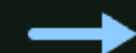
the actions in `f5_actions.py` ask

```
book.compute_total()
```

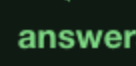
```
book.compute_by_category()
```

```
book.filter(wanted_category)
```

asks



answers



```
class ExpenseBook expense_book.py
```

```
self.expenses
```

```
def compute_total(self)
```

```
def compute_by_category(self)
```

```
def filter(self, category: str)
```

STAY AS HELPERS `is_valid_amount()` in `f1` checks the typed amount.

`format_row()` in `f2` formats one table row. Neither one needs the book's list.

One menu choice: the action asks, the book answers choice 5 (Show one category), you type food

ACTION

1. Ask the user

```
input("Category to show: ")
```



ACTION

2. Ask the book

```
book.filter(wanted_category)
```



EXPENSEBOOK

3. Find the matches

```
for expense in self.expenses
```



ACTION

4. Show the answer

```
print(format_row(expense))
```

The owner does the work

The book holds the expenses, so it answers the questions about them: the total, the spending by category, the matches.

A new screen, the same book

Now: the command line

```
f5_actions.py with input() and print()
```

Later: a web app

a web page with a form and a table



ExpenseBook

the same methods

no `input()` or `print()`

In Step-11, the **rich** package changes the screen, and `expense_book.py` stays the same.

One place for all three

They now live in `expense_book.py`:

- the total: `compute_total()`
- by category: `compute_by_category()`
- the matches: `filter()`

Before, they were in `f3`, `f4` and `f5`.

Step-05: Two **Helper Files** Move Into the **ExpenseBook**

`compute_total()` and `compute_by_category()` become **methods**, the filter moves in too, and the actions **ask the book**.

BEFORE: HELPER FILES DO THE WORK

end of Step-04

Total spent `f3_compute_total.py` in `compute_total()`

```
def compute_total(expenses: list[Expense]) -> float:
    total = 0.0
    for expense in expenses:
        total += expense.amount
```

By category `f5_actions.py` in `show_by_category()`

```
totals = compute_by_category(book.expenses)
```

One category `f5_actions.py` in `filter_by_category()`

```
matches = [
    expense
    for expense in book.expenses
    if expense.category == wanted_category
]
```

AFTER: THE BOOK DOES THE WORK

three methods in `expense_book.py`

Total spent `expense_book.py` in `compute_total()`

```
def compute_total(self) -> float:
    total = 0.0
    for expense in self.expenses:
        total += expense.amount
```

By category `f5_actions.py` in `show_by_category()`

```
totals = book.compute_by_category()
```

One category `expense_book.py` in `filter()`

```
def filter(self, category: str) -> list[Expense]:
    return [expense for expense in self.expenses if expense.category == category]
```

What changes

`expense_book.py`

three new methods: `compute_total()` `compute_by_category()` `filter()`

`f5_actions.py`

the actions ask the book: `book.compute_total()` `book.compute_by_category()` `book.filter(wanted_category)`

`f3_compute_total.py`

deleted

`f4_compute_by_category.py`

deleted

their code lives in the class now

Step-05: 5.1 Two Helpers Become Methods in `expense_book.py`

Your two helper functions move into the class as methods, with one change: `self.expenses`.

CONCEPTS USED

Classes: methods

Section 13

Accumulator: a running total

Section 06

Dictionaries: `.get()` with a default

Section 05

List comprehension: filter with `if`

Section 06

expense_book.py

`f3` and `f4` move in

`compute_total()`

moved in, same name

`compute_by_category()`

moved in, same name

`filter()`

the list comprehension, moved in

One change

`expenses` becomes `self.expenses`

WHAT WE BUILD

Step-05: 5.2 and 5.3 Actions Call the Methods in f5_actions.py

The actions call the book's methods, and the **f3** and **f4** files and their imports are deleted.

CONCEPTS USED

Modules: `from ... import`
Section 07

Classes: methods
Section 13

WHAT WE BUILD

f5_actions.py
f3 and f4 deleted

show_total()
calls `book.compute_total()`

show_by_category()
calls `book.compute_by_category()`

filter_by_category()
calls `book.filter()`

Deleted
the f3 and f4 files

Choice 3 output
Total spent: 5.50, as before

Step-06: Two New Things, `datetime` and `__repr__`

Every expense gets a **time stamp**, `created_at`, and a **readable way to show itself**.

1 A time stamp with `datetime`

The **book** stamps each expense with the time as it stores it.

```
stamp the time expense_book.py store_expense()

stamp = datetime.now().isoformat(timespec="seconds")
expense = Expense(description, amount, category, stamp)
```

`datetime.now()` the current date and time
`.isoformat(...)` turns it into text like `2026-09-16T21:50:17`

```
a fourth value expense.py __init__()

self.created_at = created_at
```

2 What is `__repr__`?

`__repr__` is a special method. It returns the **exact text Python shows for an object**: at the `>>>` prompt, in lists, and in `print()` when there is no `__str__`.

```
show every value expense.py __repr__()

def __repr__(self) -> str:
    return (f"Expense(description={self.description!r}, amount={self.amount!r}, "
            f"category={self.category!r}, created_at={self.created_at!r})")
```

`!r` puts quotes around text values, like `'Coffee'`

At a Python prompt the same line, without and with `__repr__`

```
>>> book.store_expense("Coffee", 3.50, "food")
```

WITHOUT `__REPR__` `<expense.Expense object at 0x100e10980>` only the file and class name, and a memory address

WITH `__REPR__` `Expense(description='Coffee', amount=3.5, category='food', created_at='2026-09-16T21:50:17')`

Why a time stamp?

Every expense remembers when it was added. In Step-07, `created_at` is saved to the file with the other three values.

Why `__repr__`?

Without it, every expense shows up as `<expense.Expense object at 0x...>`, with no values at all. With it, you **see your data** everywhere Python shows an expense, so checking your work and fixing bugs gets easy.

Step-06: 6.1 The Fourth Attribute and `__repr__` in `expense.py`

Every expense gets a `created_at` attribute and a `__repr__` that shows all four values.

CONCEPTS USED

Classes: `__init__` and `self`
Section 13

Classes: `__repr__`
Section 13

f-strings: `!r`
New in this project

`expense.py`
finished in this step

WHAT WE BUILD

`created_at`
the fourth attribute

`__repr__()`
shows all four values

`!r`
quotes around text values

`expense.py`
finished in this step

Step-06: 6.2 and 6.3 Stamp the Time in `expense_book.py`

`store_expense()` stamps each new expense with the time, and a Python prompt shows it through `__repr__`.

CONCEPTS USED

Dates: `datetime`
Section 11

Dates: `.isoformat()`
New in this project

`expense_book.py`
and the finished f5

`store_expense()`
stamps the time

`datetime.now()`
the current date and time

`.isoformat()`
text like 2026-09-16T12:43:14

`f5_actions.py`
finished in this step

At a Python prompt
`book.store_expense()` returns an `Expense(...)`

WHAT WE BUILD

Step-07: Save Your Expenses to a JSON File

The book gets a file path and a `save()` method, and choice 6 becomes **Save and quit**.

BEFORE

The expenses live only in memory, in `self.expenses`. Quit, and they are gone.

AFTER

Choice 6 writes them to `data/expenses.json`, next to your code.

How `save()` writes the file `expense_book.py`

1. IN MEMORY

Expense objects
`self.expenses`



2. ONE DICTIONARY EACH

object to dict
`"amount": expense.amount`



3. JSON TEXT

dicts to text
`json.dumps(rows, indent=2)`



4. THE FILE

text to the file
`write_text()`

WHY DICTS? `json.dumps` can only write **plain data** like dictionaries and lists, not your own `Expense` objects.

what you see choice 6 in the menu

```
6) Save and quit
Choose 1-6: 6
Saved. Goodbye!
```

The expenses are now in a file, next to `main.py`. The full code changes are on the next slide.

`data/expenses.json` after adding Coffee, then choice 6

```
[
  {
    "description": "Coffee",
    "amount": 3.5,
    "category": "food",
    "created_at": "2026-09-16T21:50:17"
  }
]
```

NEXT Step-08 loads this file back when the app starts, so your expenses **survive a restart**.

Step-07: The Code Changes in `expense_book.py` and `main.py`

+ new line - removed line grey lines stay the same

`expense_book.py` the book gets a file path and a `save()` method

```
+ import json
+ from datetime import datetime
+ from pathlib import Path
...
- def __init__(self) -> None:
+ def __init__(self, path: Path) -> None:
+     self.path = path
+     self.expenses: list[Expense] = []
```

```
...
+ def save(self) -> None:
+     """Write every expense to the JSON file, making the data folder if it is missing."""
+     rows = [{"description": expense.description, "amount": expense.amount,
+              "category": expense.category, "created_at": expense.created_at}
+             for expense in self.expenses]
+     self.path.parent.mkdir(parents=True, exist_ok=True)
+     self.path.write_text(json.dumps(rows, indent=2))
```

`mkdir(parents=True, exist_ok=True)` makes the `data` folder, and does nothing if it already exists. `indent=2` makes the file easy to read.

`main.py` the book gets its file, and choice 6 saves

```
+ from pathlib import Path
+ from expense_book import ExpenseBook
...
+ print("5) Show one category")
- print("6) Quit")
+ print("6) Save and quit")
```

```
def main() -> None:
- book = ExpenseBook()
+ book = ExpenseBook(Path(__file__).parent / "data" / "expenses.json")
+ while True:
...
+     elif choice == "6":
-         print("Goodbye!")
+         book.save()
+         print("Saved. Goodbye!")
+         break
```

`Path(__file__).parent` is the folder that holds `main.py`, so the file always lands next to your code.

Step-07: 7.1 Save to JSON in `expense_book.py`

`save()` turns every expense back into a dict and writes them all to one JSON file.

CONCEPTS USED

JSON: `json.dumps`
Section 10

Files: `Path` and `write_text()`
Section 10

Paths: `mkdir()`
Section 10

List comprehension: `[... for ...]`
Section 06

`expense_book.py`
`save()`

WHAT WE BUILD

`save()`
writes every expense as JSON

`self.path`
the file the book saves to

One dict per expense
plain data that `json.dumps` can write

`indent=2`
a file that is easy to read

Step-07: 7.2 The Data Path and Choice 6 in `main.py`

`main()` gives the book a file next to `main.py`, and choice 6 saves before it quits.

CONCEPTS USED

Paths: `__file__`
Section 10

Classes: methods
Section 13

WHAT WE BUILD

`Path(__file__).parent`
the folder that holds `main.py`

`data/expenses.json`
next to `main.py`

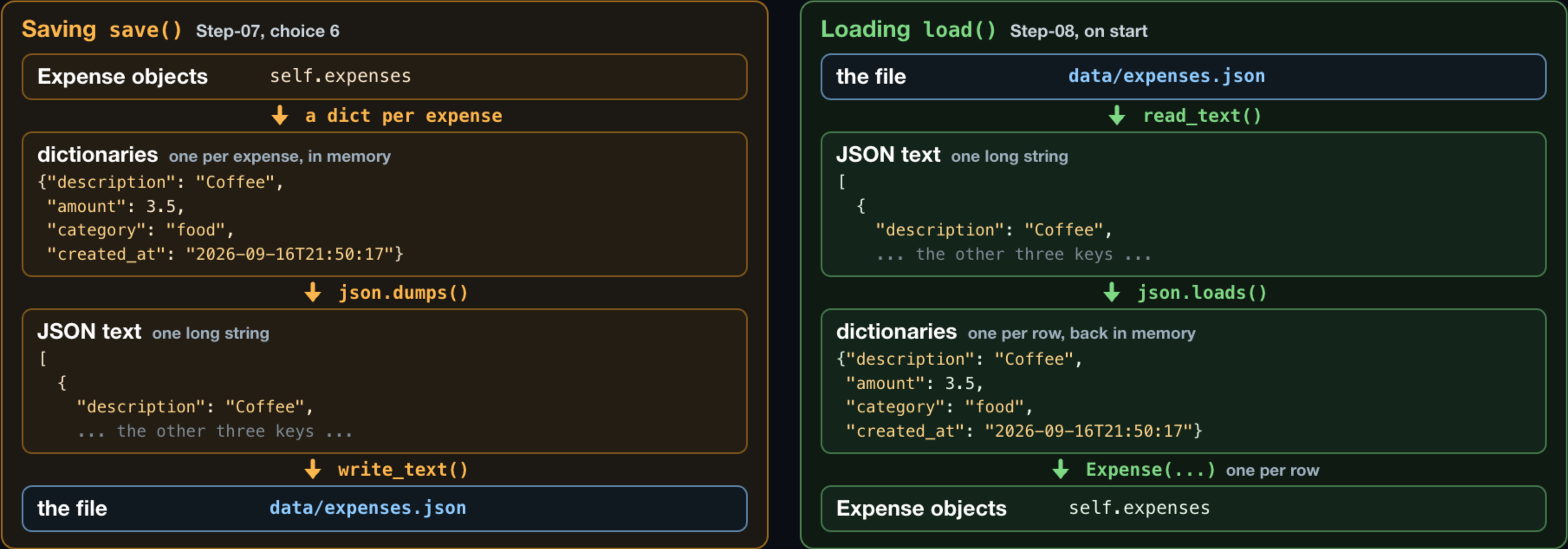
Menu line 6
Save and quit

Choice 6
saves, then prints `Saved. Goodbye!`

`main.py`
Save and quit

Step-08: The Same Trip, Out and Back

`save()` turns your objects into text. `load()` turns that text back into objects.



WHY THE MIDDLE STEPS? A file holds **text**, and JSON only knows plain data: text, numbers, lists and dictionaries. Your **Expense** objects go out as dictionaries and come back as objects, and the same four keys travel both ways: **description amount category created_at**

Step-08: Load It Back, So Your Data Survives a Restart

A `load()` method reads the file into **Expense** objects, and `main()` calls it once, before the menu loop.

BEFORE

Step-07 writes the file, but nobody reads it. Start the app again and the book is **empty**.

AFTER

`book.load()` fills the book from the file before the menu appears.

How `load()` reads the file `expense_book.py`

1. THE FILE

JSON text on disk
`data/expenses.json`



2. TEXT TO DICTS

read and parse
`json.loads(self.path.read_text())`



3. DICTS TO OBJECTS

one Expense per row
`for row in rows`



4. IN THE BOOK

ready for the menu
`self.expenses`

FIRST RUN? There is no file yet, so `if not self.path.exists()` starts with an **empty list** and returns. That is normal, not an error.

the new method `expense_book.py`

```
def load(self) -> None:
    if not self.path.exists():
        self.expenses = []
        return
    rows = json.loads(self.path.read_text())
    self.expenses = [Expense(row["description"], row["amount"],
                             row["category"], row["created_at"]) for row in rows]
```

run it again after Save and quit

Choose 1-6: 2

Description	Category	Amount
Coffee	food	3.50

The expense is **still there**. Milestone 1 lost it on restart.

one line in `main.py` `book.load()` right after the book is created, and **before** the menu loop starts

Step-08: 8.1 Load It Back in `expense_book.py`

`load()` reads the JSON file back into Expense objects, or starts empty when there is no file.

CONCEPTS USED

JSON: `json.loads`
Section 10

Files: `read_text()` and `.exists()`
Section 10

List comprehension: `[... for ...]`
Section 06

`expense_book.py`
`load()`

WHAT WE BUILD

`load()`
reads the file into Expense objects

Missing file
an empty book, and no error

One Expense per dict
built by a list comprehension

`expense_book.py`
finished in this step

Step-08: 8.2 Load on Start in `main.py`

`main()` calls `book.load()` before the menu loop, so your data survives a restart.

CONCEPTS USED

Classes: methods

Section 13

Loops: while True

Section 06

WHAT WE BUILD

`main()`

calls `book.load()` on start

Before the loop

the book fills from the file

Second run

lists the expense you saved

`main.py`

one new line

Step-09: A Broken File Must Not Crash Your App

One **try / except** around **book.load()** turns a crash into a **warning**, and the app starts empty.

break it on purpose `data/expenses.json`

```
[
  {
    "description": "Coffee",
    "amount": 3.5,
```

before the fix the app dies before the menu

```
File ".../04-livecode-build-incremental/main.py", line 26, in main
    book.load()
File ".../04-livecode-build-incremental/expense_book.py", line 54, in load
    rows = json.loads(self.path.read_text())
json.decoder.JSONDecodeError: Expecting property name enclosed in double quotes
```

the fix `main.py` in `main()`

```
try:
    book.load()
except (json.JSONDecodeError, TypeError, KeyError):
    print("Warning: data file is corrupt; starting empty.")
    book.expenses = []
```

after the fix same broken file

Warning: data file is corrupt; starting empty.

Choose 1-6: 2
No expenses yet.

Three errors, one reply **except (A, B, C)**

json.JSONDecodeError

the text is not valid JSON

TypeError

valid JSON, wrong shape, like `[1, 2, 3]`

KeyError

a record is missing a field

MISSING OR BROKEN?

A **missing** file is normal: **load()** handles it with an **if** and starts empty. A **broken** file raises an error, so **main()** catches it.

Step-09: Run It or Import It, the `__main__` Guard

Two lines decide whether the menu starts: `if __name__ == "__main__":` and an indented `main()`.

before the last line of `main.py`

```
main()
```

after the last two lines of `main.py`

```
if __name__ == "__main__":  
    main()
```

You run it `python3 main.py`

`__name__` is "`__main__`" in this file

the `if` is **true**

`main()` runs

The menu starts

Something imports it `import main`

`__name__` is "`main`" instead

the `if` is **false**

`main()` does not run

You get the functions, with no menu

Try it at a Python prompt in your build folder, type `python3`

```
>>> import main           nothing is printed: no menu, no prompt  
>>> main.show_menu()     you call it yourself, and the menu appears
```

```
==== Expense Tracker 2.0 ====
```

```
1) Add expense
```

The same file can now be **a program to run** and **a module to import**.

FROM SECTION 12 You met this guard with a small package. Here it goes into **your own app**, so `main.py` can be run or imported.

Step-09: 9.1 Do Not Crash on a Broken File in `main.py`

A `try / except` around `load()` turns a broken data file into a warning instead of a crash.

CONCEPTS USED

Error handling: `try / except`
Section 09

Error handling: `except (A, B)`
Section 09

Exceptions: `json.JSONDecodeError`
New in this project



`main.py`
the corrupt-file guard

WHAT WE BUILD

`try around load()`
catches three errors

Broken file
a warning, then an empty book

Printed warning
Warning: data file is corrupt; starting empty.

Choice 6
saves the empty book over the broken file

Step-09: 9.2 Run It or Import It in `main.py`

`if __name__ == "__main__":` starts the menu when you run the file, and stays quiet when another file imports it.

CONCEPTS USED

The `__main__` guard
Section 12

Modules: `import`
Section 07

`main.py`
the `__main__` guard

WHAT WE BUILD

`if __name__ == "__main__":`
the last two lines of `main.py`

`python3 main.py`
the menu starts

`import main`
prints nothing at all

`main.show_menu()`
you call it yourself

`main.py`
finished in this step

Step-10: Run It All in **main.py**

Run every menu choice, quit and restart to see your expenses come back, then watch a **broken** data file warn instead of crash.

CONCEPTS USED

Run a program: python3
Section 02

Modules: from ... import
Section 07

Classes: class
Section 13

Files: write_text(), read_text()
Section 10

Error handling: try / except
Section 09



WHAT WE BUILD

Choices 1 to 6
all six work in one run

Total spent: 17.50
your three expenses, added up

data/expenses.json
written by choice 6

Restart
your expenses load again

Broken file
a warning, not a crash

Step-11 (bonus): One **venv**, One Package, a Real Table

Optional, and in its own folder: Section 12 meets your own app. **expense_book.py** and **main.py** do not change at all.

set it up in 05-livecode-build-incremental-with-rich/

```
cd ..
cd 05-livecode-build-incremental-with-rich
python3 -m venv .venv
source .venv/bin/activate
pip install rich
```

On Windows the **source** line is **.venv\Scripts\activate** instead. Your prompt then starts with **(.venv)**, so you can see the toolbox is on.

write down what the project needs **requirements.txt**

```
pip freeze > requirements.txt
```

rich==15.0.0 and the packages rich itself needs. Anyone who clones the project runs **pip install -r requirements.txt**. Add **.venv/** to **.gitignore**: it is rebuilt, never committed.

if you skip the venv the app stops before the menu

```
File ".../f5_actions.py", line 3, in <module>
    from rich.console import Console
ModuleNotFoundError: No module named 'rich'
```

Only the part that talks to the screen changes

CHANGES

f2_format_row.py a new make_table()

CHANGES

f5_actions.py prints one table

NO CHANGE

expense.py

expense_book.py

f1_is_valid_amount.py

main.py

ONE TOOLBOX PER PROJECT

Turn it off again with **deactivate** and **rich** is gone: it lives only inside **this folder's .venv**, never in your system Python.

Step-11 (bonus): A Table Instead of Printed Lines

One new helper, `make_table()`, and the two actions print one table instead of a header and a row each.

the new helper `f2_format_row.py`

```
from rich.table import Table
...
def make_table(expenses: list[Expense]) -> Table:
    table = Table(title="Expenses")
    table.add_column("Description")
    table.add_column("Category")
    table.add_column("Amount", justify="right")
    for expense in expenses:
        table.add_row(expense.description, expense.category,
                      f"{expense.amount:,.2f}")
    return table
```

print it `f5_actions.py` `list_expenses()` and `filter_by_category()`

```
from f1_is_valid_amount import is_valid_amount
from f2_format_row import format_row
from rich.console import Console

from f2_format_row import make_table
...
print(f"{'Description':<20}{'Category':<14}{'Amount':>10}")
for expense in book.expenses:
    print(format_row(expense))

Console().print(make_table(book.expenses))
```

before: choice 2, printed lines

Description	Category	Amount
Coffee	food	3.50
Bus	transport	2.00

after: choice 2, a rich table

Expenses		
Description	Category	Amount
Coffee	food	3.50
Bus	transport	2.00

THE BOOK NEVER CHANGED

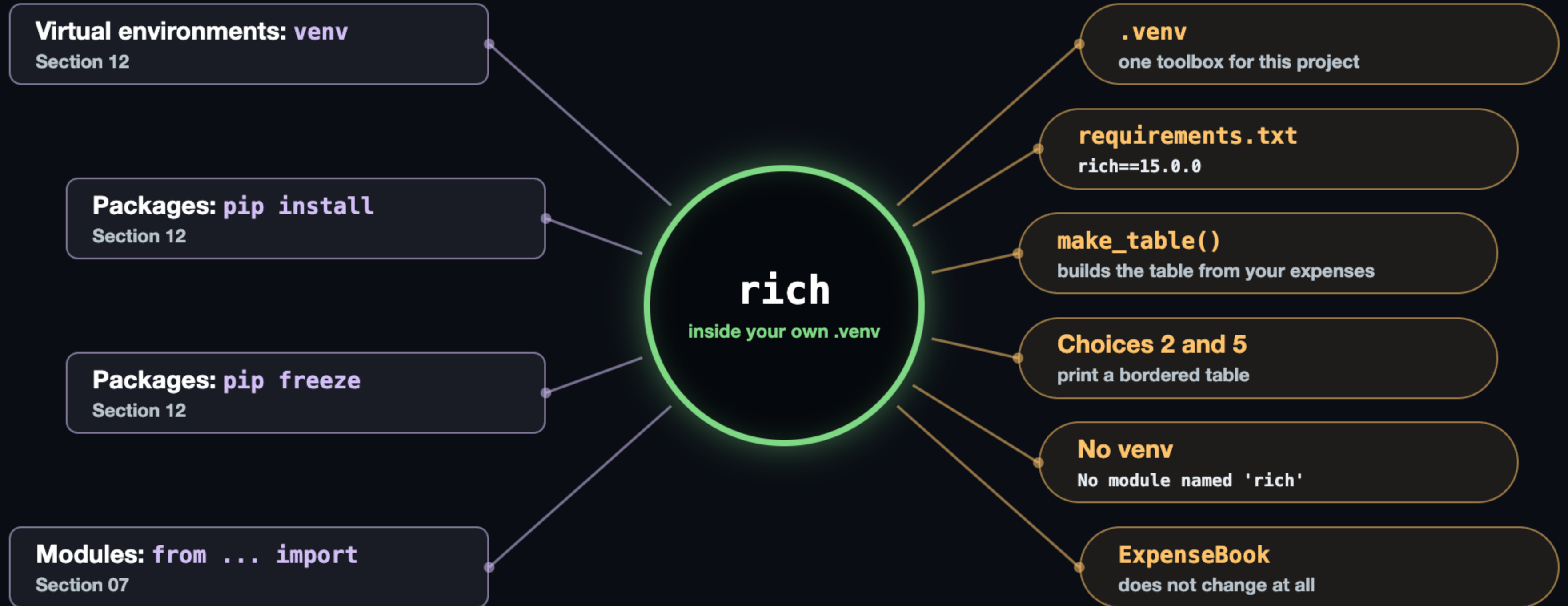
The same **ExpenseBook** answers the same questions. Only the screen code changed, which is exactly why **Step-05** moved the work into the book.

Step-11: 11.1 to 11.4 Your First Installed Package: rich

A virtual environment, one **pip install**, and the expense list arrives as a real table.

CONCEPTS USED

WHAT WE BUILD



Project 2: What You Built

One tracker, six files, and the ideas from Sections 07 to 13 working together. Your data is still there after you quit.

the six files, and the job each one has

`expense.py` one expense, an object with four attributes and a `__repr__`

`expense_book.py` the book: holds the list, and does the work on it

`f1_is_valid_amount.py` checks the amount the user typed

`f2_format_row.py` turns one expense into one aligned row

`f5_actions.py` the five actions behind the menu

`main.py` the menu, the loop, the save on quit, the guard

what it does when you run it

- 1 Adds an expense, stamped with `created_at`
- 2 Lists them all, one aligned row each
- 3 Totals the spending with `book.compute_total()`
- 4 Totals it per category
- 5 Shows one category with `book.filter()`
- 6 Saves to `data/expenses.json` and quits, then **loads it back on the next run**

The ideas you used, and where each one came from

13 classes, `__init__`, `self`, methods

08 type hints on every function

10 `Path`, `write_text`, `read_text`, `json`

09 `try / except`, and `except (A, B)`

11 `datetime` and `.isoformat()`

07 your own modules, `from ... import`

12 the `__main__` guard, and `venv` and `pip` in the bonus

Where this goes next

READY FOR MORE Your `ExpenseBook` has no `input()` and no `print()`, so the same class can sit behind a **web API**

Next in the StackSimplify family, and this project is the foundation they build on

Complete Python Bootcamp

Python for Machine Learning

The MLOps Courses